

SecFlow
Phase 1 - Repository Analysis Service
Tools, MCP Contracts & Python Integration Reference

Purpose

tool description, وما الذي يراه الـ Phase 1، داخل external/open-source tool وثيقة مرجعية للفريق توضح كل واضح ومفهوم لها Tool الـ الهدف هو أن تكون كل interface لها Tool الـ MCP contract، والـ Input/Output، والـ CLI/Python integration. الـ upstream capabilities بدون خلط بين الـ Agent، الـ SecFlow wrappers.

Layer	Responsibility
Repository Service	Fetch/clone, workspace management, preprocessing, tool execution, normalized snapshot
AI Analyzer	Reasoning, selective tool calls, evidence-based interpretation, JSON Manifest
Contract	Structured inputs/outputs; tools are deterministic where possible; AI owns interpretation

Approved Phase 1 Toolchain

Tool	Primary purpose	AI exposed?
Git	Clone + repository metadata + file enumeration	No - internal deterministic
scc	Language/LOC/complexity + hotspots/coupling	Yes - direct MCP or SecFlow wrapper
Tree-sitter	AST / syntax structure / code symbols	Yes - SecFlow wrapper
Syft	Dependency/package inventory & SBOM	Yes - SecFlow wrapper
Semgrep	Pattern/static analysis + security evidence	Yes - SecFlow wrapper / Semgrep MCP
detect-secrets	Secret-like finding detection	Yes - SecFlow wrapper

Design rule

The AI should not receive a generic shell executor. It receives narrow, read-only, purpose-built tools. Git clone remains internal to the Repository Service; AI tools inspect the prepared workspace/snapshot only.

1. Tool Architecture & Conventions

```

GitHub URL
↓
Repository Service (FastAPI)
↓
Clone + workspace + filtering
↓
Open-source tools
├─ Git
├─ scc
├─ Tree-sitter
├─ Syft
├─ Semgrep
└─ detect-secrets
↓
Normalized tool results
↓
repository_snapshot.yaml
↓
AI Analyzer / MCP tools
↓
JSON Manifest
    
```

1.1 Standard Tool Interface

Every SecFlow Tool should define

1) name, 2) AI-facing description, 3) when to use, 4) input schema, 5) output schema, 6) constraints/limitations, 7) evidence semantics. Python type hints + docstrings can be used to generate MCP metadata, but the contract should remain explicit and testable.

1.2 Common input model

```

class ToolContext(BaseModel):
    repository_path: str
    include_paths: list[str] = []
    exclude_paths: list[str] = []
    timeout_seconds: int = 60
    max_output_items: int = 500
    
```

1.3 Common output model

```

{
  "tool": "...",
  "status": "success | partial | error",
  "evidence": [...],
  "data": {...},
  "warnings": [...]
}
    
```

Important

The tool should return evidence paths and structured facts. It should not invent architecture conclusions. Architecture inference belongs to the AI Analyzer.

2. Git - Repository Fetch & Metadata

Field	Value
Role in SecFlow	Repository lifecycle: clone a user repository into an isolated workspace and expose deterministic metadata.
AI exposure	Internal only. Not exposed as a generic AI shell tool.

1. AI-facing Description

Description passed to the model

Clone the requested Git repository into the prepared workspace and return deterministic repository metadata. Use it to obtain the source tree and repository facts; do not use it to execute arbitrary commands.

2. When to Use / When NOT to Use

Use when	Do not use when
starting a repository analysis; creating a disposable workspace; retrieving branch/commit/file metadata.	source-code interpretation, architecture reasoning, security findings, or arbitrary shell execution.

3. Input / Output Contract

Input	Output
repo_url: string; optional branch/ref; depth policy; workspace path.	clone status, resolved commit/ref, repository root, tracked file list, basic git metadata.

4. MCP Tool Contract (SecFlow-facing)

Internal Python service function - not an MCP tool

```
def clone_repository(repo_url: str, workspace: str, ref: str | None = None, depth: int = 1) -> dict:
```

```
...
```

5. CLI

```
git clone --depth=1 <REPO_URL> <WORKSPACE>
```

```
git -C <WORKSPACE> rev-parse HEAD
```

```
git -C <WORKSPACE> ls-files -z
```

6. Python Wrapper

```
import subprocess
```

```
def run_git(repo_url: str, workspace: str):
    subprocess.run(
        ["git", "clone", "--depth=1", repo_url, workspace],
        check=True, timeout=120
    )
```

7. Limits / Security

- Use argument arrays, never `shell=True` for untrusted values.
- Apply network/time/output limits.
- Use disposable workspaces and guaranteed cleanup.
- Do not expose arbitrary command execution to the model.
- Do not execute repository code during clone/preprocessing.

Official reference: Git official documentation: git-clone

3. scc - Codebase Metrics, Hotspots & Coupling

Field	Value
Role in SecFlow	Deterministic source-code statistics and repository-history signals.
AI exposure	Yes. It can be used directly through the upstream MCP server or wrapped behind SecFlow.

1. AI-facing Description

Description passed to the model

Analyze a repository to determine language composition, lines of code, complexity and selected history signals. Use the results as quantitative evidence; do not treat them as a complete architecture description.

2. When to Use / When NOT to Use

Use when	Do not use when
you need language detection, code-size metrics, complexity, hotspots, or change-coupling evidence.	file content explanation, dependency resolution, secret scanning, or semantic architecture inference.

3. Input / Output Contract

Input	Output
path; for MCP hotspots/coupling also depth/limit and optional target file.	JSON metrics, per-language/per-file statistics, complexity; hotspots and coupling when requested.

4. MCP Tool Contract (SecFlow-facing)

```
@mcp.tool()
def scc_analyze(path: str, locomo_preset: str = "medium") -> dict:
    """Analyze a repository using scc and return deterministic code metrics."""
    ...

@mcp.tool()
def scc_hotspots(path: str, depth: int = 1000, limit: int = 50) -> dict:
    """Rank repository files by complexity/change hotspot score."""
    ...

@mcp.tool()
def scc_coupling(path: str, file: str | None = None, depth: int = 1000, limit: int = 50) -> dict:
    """Report change-coupling relationships from Git history."""
    ...
```

5. CLI

```
scc --format json <REPO_PATH>
scc --format json --by-file <REPO_PATH>
scc --coupling <REPO_PATH>
scc --coupling-for <FILE>
scc --mcp
```

6. Python Wrapper

```
import subprocess, json

def scc_json(repo: str) -> dict:
    p = subprocess.run(["scc", "--format", "json", repo],
        check=True, capture_output=True, text=True, timeout=60)
    return json.loads(p.stdout)
```

7. Limits / Security

- Require the target path to stay inside the prepared repository workspace.
- Do not allow the AI to change arbitrary scc flags unless explicitly whitelisted.
- History-based tools require a Git repository and may depend on clone depth.
- Large repositories may need timeout/output limits.

Official reference: scc official README documents --mcp and the analyze/hotspots/coupling MCP tools.

4. Tree-sitter - Syntax / AST Analysis

Field	Value
Role in SecFlow	Parse source files into syntax trees and expose structured code facts.
AI exposure	Yes - through a SecFlow MCP wrapper; upstream library is a parser, not the SecFlow MCP interface.

1. AI-facing Description

Description passed to the model

Parse selected source files with the appropriate Tree-sitter language grammar and return structured syntax information such as functions, classes, imports, decorators and other nodes. Use it for deterministic structural evidence.

2. When to Use / When NOT to Use

Use when	Do not use when
you need code structure, symbols, imports, decorators, function/class inventory, or syntax-level evidence.	dependency inventory, secret scanning, full architecture decisions, or dynamic behavior.

3. Input / Output Contract

Input	Output
file_path; language/grammar; optional query/pattern; max nodes.	language, parse status, root node, selected nodes/symbols, source ranges, warnings.

4. MCP Tool Contract (SecFlow-facing)

```
@mcp.tool()
def parse_source_file(file_path: str, language: str, max_nodes: int = 500) -> dict:
    """Parse one source file and return structured AST evidence."""
    ...

@mcp.tool()
def find_code_patterns(file_path: str, language: str, pattern: str) -> dict:
    """Run a bounded Tree-sitter query and return matching nodes."""
    ...
```

5. CLI

```
# Tree-sitter is primarily a Python library here.
# CLI grammar work is separate; SecFlow calls the Python binding.
python -c "from tree_sitter import Parser; print('ok')"
```

6. Python Wrapper

```
from tree_sitter import Language, Parser

def parse_source(source: bytes, language: Language):
    parser = Parser(language)
    tree = parser.parse(source)
    return tree.root_node
```

7. Limits / Security

- Keep parsing read-only and bounded by file size/node count.
- Do not execute source code to understand it.
- Language grammar must be explicitly selected/validated.
- Return source ranges and evidence paths so the AI can cite where a fact came from.

Official reference: Tree-sitter official parser guide and py-tree-sitter documentation.

5. Syft - Dependency / Package Inventory

Field	Value
Role in SecFlow	Generate a structured inventory/SBOM-like view of packages discovered in the repository.
AI exposure	Yes - through a SecFlow wrapper. Do not expose arbitrary Syft targets/flags to the model.

1. AI-facing Description

Description passed to the model

Inspect a repository or supported artifact and return the packages/dependencies discovered by Syft. Use this deterministic inventory as evidence for technology and dependency identification.

2. When to Use / When NOT to Use

Use when	Do not use when
you need package names, versions, ecosystems, or SBOM-style evidence.	source-code structure, secrets, or security policy conclusions by itself.

3. Input / Output Contract

Input	Output
source directory; output format; optional cataloger/package-type options from an allowlist.	structured package/artifact records; optional JSON SBOM document; source metadata.

4. MCP Tool Contract (SecFlow-facing)

```
@mcp.tool()
def inventory_dependencies(repository_path: str, max_items: int = 1000) -> dict:
    """Discover package/dependency artifacts in a prepared repository."""
    ...
```

5. CLI

```
syft <REPO_PATH> -o json
# Example: write SBOM artifact
syft <REPO_PATH> -o json=sbom.json
```

6. Python Wrapper

```
import subprocess, json

def syft_inventory(repo: str) -> dict:
    p = subprocess.run(["syft", repo, "-o", "json"],
        check=True, capture_output=True, text=True, timeout=120)
    return json.loads(p.stdout)
```

7. Limits / Security

- Run against the prepared repository only.
- Treat package metadata as evidence, not proof that code is actually executed in production.
- Large repositories can produce large SBOMs; normalize and cap output before model ingestion.
- Keep the raw SBOM as an artifact when traceability matters.

Official reference: Anchore Syft JSON schema/reference documents discovered artifacts/packages and source metadata.

6. Semgrep - Pattern & Security Analysis

Field	Value
-------	-------

Role in SecFlow	Run bounded static analysis and return structured findings that can be used as evidence.
AI exposure	Yes. Semgrep provides an MCP integration; SecFlow can also expose a narrow wrapper around the CLI.

1. AI-facing Description

Description passed to the model

Run approved Semgrep rules/packs against the prepared repository and return findings with rule IDs, severities, paths and source locations. Use it for security/pattern evidence, not for generic repository discovery.

2. When to Use / When NOT to Use

Use when	Do not use when
you need security findings, source-pattern detection, or evidence for a technology/security claim that maps cleanly to a rule.	broad architecture inference, dependency inventory, repository cloning, or unrestricted rule/config execution.

3. Input / Output Contract

Input	Output
repository path; approved rules/config identifier; optional language include/exclude filters.	structured findings including check/rule, severity, path, line/column, message and metadata.

4. MCP Tool Contract (SecFlow-facing)

```
@mcp.tool()
def semgrep_scan(repository_path: str, config: str = "auto", max_findings: int = 200) -> dict:
    """Run an allowlisted Semgrep scan and return normalized findings."""
    ...
```

5. CLI

```
semgrep scan --config auto --json <REPO_PATH>
# Prefer an allowlisted config for deterministic Phase 1 behavior
semgrep scan --config <ALLOWED_CONFIG> --json <REPO_PATH>
```

6. Python Wrapper

```
import subprocess, json

def semgrep_scan(repo: str, config: str = "auto") -> dict:
    p = subprocess.run(["semgrep", "scan", "--config", config, "--json", repo],
                        check=False, capture_output=True, text=True, timeout=120)
    return json.loads(p.stdout or "{}")
```

7. Limits / Security

- Do not let the model pass arbitrary local config paths or shell snippets.
- Run in an isolated workspace/container with network disabled where possible.
- Normalize only the fields needed by the agent.
- Treat findings as evidence; severity and remediation remain policy-dependent.

Official reference: Semgrep publishes an MCP integration for agents and documents security considerations for MCP deployments.

7. detect-secrets - Secret-like Finding Detection

Field	Value
Role in SecFlow	Identify likely secrets using plugin/filter based heuristics and return

	machine-readable findings.
AI exposure	Yes - through a SecFlow wrapper; direct upstream CLI is preferred for deterministic behavior.

1. AI-facing Description

Description passed to the model

Scan the prepared repository for likely secret values and return a baseline/report suitable for triage. Use it as a signal only; it is not a guarantee that all secrets are found.

2. When to Use / When NOT to Use

Use when	Do not use when
you need to detect API-key-like strings, tokens, private keys, passwords and other secret-shaped content.	architecture reasoning, dependency analysis, or runtime secret validation.

3. Input / Output Contract

Input	Output
repository path; optional baseline/plugin/filter policy controlled by SecFlow.	baseline/report JSON containing detector, file, line and finding metadata; secret values should be redacted from model output.

4. MCP Tool Contract (SecFlow-facing)

```
@mcp.tool()
def detect_secrets_scan(repository_path: str, max_findings: int = 200) -> dict:
    """Scan a prepared repository for likely secrets and return redacted findings."""
    ...
```

5. CLI

```
detect-secrets scan <REPO_PATH> --all-files > .secrets.baseline
detect-secrets audit .secrets.baseline --report --json
```

6. Python Wrapper

```
from detect_secrets import SecretsCollection
from detect_secrets.settings import default_settings

def scan_file(path: str) -> dict:
    secrets = SecretsCollection()
    with default_settings():
        secrets.scan_file(path)
    return secrets.json()
```

7. Limits / Security

- Never return the actual secret value to the LLM unless there is a separately approved workflow.
- Use allowlists/filters to control false positives.
- A no-finding result does not prove that the repository has no secrets.
- Keep the detailed baseline/report as a restricted artifact.

Official reference: [Yelp detect-secrets official README documents scan, audit, plugins/filters, and Python usage.](#)

8. SecFlow Tool Orchestrator

The service should own one top-level deterministic orchestration function. It calls approved tools, normalizes their outputs, and builds the YAML snapshot. The AI is not responsible for cloning or arbitrary shell commands.

```
def build_repository_snapshot(repo_url: str, config: AnalysisConfig) -> str:
    workspace = clone_repository(repo_url, config.workspace)
    repo_meta = collect_repo_metadata(workspace)
    metrics = scc_json(workspace)
    deps = syft_inventory(workspace)
    tree = parse_selected_files(workspace, config)
    security = semgrep_scan(workspace, config.semgrep_profile)
    secrets = detect_secrets_scan(workspace)

    snapshot = normalize(
        repo_meta=repo_meta,
        metrics=metrics,
        dependencies=deps,
        structure=tree,
        security=security,
        secrets=redact(secrets),
    )
    return write_yaml(snapshot)
```

8.1 Normalized Snapshot

```
repository:
  name: sample-app
  url: https://github.com/example/sample-app
  commit: <resolved-sha>

structure:
  files: [...]
  languages: [...]

metrics:
  loc: ...
  complexity: ...
  hotspots: [...]

dependencies:
  ecosystems: [...]
  packages: [...]

syntax_evidence:
  imports: [...]
  functions: [...]
  classes: [...]

security_evidence:
  semgrep: [...]

secret_findings:
  count: ...
  findings: [redacted]
```

8.2 Agent-facing Tool Rules

- Tool descriptions must explain purpose, when to use, and what evidence the output means.
- Inputs must be strongly typed and bounded. Prefer enums and allowlists over free-form command strings.
- Outputs should be structured JSON with stable keys and evidence paths.
- The agent should prefer deterministic evidence before inference.
- Generic shell execution is out of scope for Phase 1.

9. Example MCP Definitions

The detailed tool sections above are the source of truth. This section shows the compact MCP metadata shape the AI should receive.

```
{
  "name": "scc_analyze",
  "description": "Analyze a prepared repository with scc. Use for language composition, LOC, comments, blanks, and complexity metrics. Returns deterministic evidence; do not use it as the sole source for architecture conclusions.",
  "inputSchema": {
    "type": "object",
    "properties": {
      "path": {"type": "string"},
      "locomo_preset": {"type": "string", "enum": ["large", "medium", "small", "local"]}
    },
    "required": ["path"]
  }
}
```

Tool	AI-facing description focus	Primary input
scc_analyze / scc_hotspots / scc_coupling	Use for deterministic metrics and Git-history signals; return evidence, not architecture conclusions.	path; optional depth/limit/file
parse_source_file / find_code_patterns	Use for AST/syntax evidence such as functions, classes, imports and decorators.	file_path; language; query/pattern
inventory_dependencies	Use for package names, versions, ecosystems and SBOM evidence.	repository_path; max_items
semgrep_scan	Use for allowlisted security/pattern rules and line-level findings.	repository_path; approved config; max_findings
detect_secrets_scan	Use for likely secret-like findings; results are redacted signals, not proof of absence.	repository_path; max_findings

10. Ownership & Team Split

Team	Member	Ownership
Team 1 - Repository Analysis / Backend	M1	FastAPI + Git + repository lifecycle
	M2	Open-source analysis tool wrappers (scc, Tree-sitter, Syft, Semgrep, detect-secrets)
	M3	Snapshot schema + security boundaries + integration
Team 2 - AI Analyzer	M4	AI Agent + orchestration + model/tool calling
	M5	Prompt + JSON Manifest schema + evidence rules
	M6	Evaluation + integration + test projects + output validation

10.1 Boundary

TEAM 1

GitHub URL

↓

clone / preprocess / tools

↓

repository_snapshot.yaml

|

—— CONTRACT ——

↓

TEAM 2

AI Agent + selective MCP tool usage

↓

JSON Manifest

Key rule

Team 1 owns the repository data plane. Team 2 owns reasoning and manifest generation. This separation lets both teams work in parallel using a mock snapshot contract.

11. Implementation Checklist

- ☐ FastAPI service created and containerized.
- ☐ Git clone uses isolated workspace + timeouts + cleanup.
- ☐ Tool wrappers use subprocess argument arrays and no shell interpolation.
- ☐ scc output normalized to stable JSON fields.
- ☐ Tree-sitter parser coverage defined for selected languages.
- ☐ Syft dependency inventory normalized.
- ☐ Semgrep profile/config allowlisted.
- ☐ detect-secrets findings redacted before model exposure.
- ☐ All tool outputs include evidence paths and warnings.
- ☐ repository_snapshot.yaml schema frozen before full integration.
- ☐ MCP tool descriptions reviewed from the AI perspective.
- ☐ Each tool has unit tests + one representative sample project.

Phase 1 acceptance flow

A small sample GitHub project is enough for the first integration. The acceptance test is: URL → isolated clone → deterministic tool results → normalized YAML snapshot → AI reads/selects evidence → validated JSON Manifest.

12. Official References

Git clone: <https://git-scm.com/docs/git-clone>

scc README / MCP: <https://github.com/boyter/scc/blob/master/README.md>

Tree-sitter parsers: <https://tree-sitter.github.io/tree-sitter/using-parsers/>

py-tree-sitter: <https://tree-sitter.github.io/py-tree-sitter/>

Syft JSON schema: <https://oss.anchore.com/docs/reference/syft/json/16/>

Semgrep MCP security guide: <https://semgrep.dev/blog/2025/a-security-engineers-guide-to-mcp/>

Semgrep MCP announcement: <https://semgrep.dev/blog/2025/giving-appsec-a-seat-at-the-vibe-coding-table/>

detect-secrets: <https://github.com/Yelp/detect-secrets>

Versioning note

External tool capabilities and MCP interfaces can change. Pin tool versions in the SecFlow build and treat this document as the Phase 1 contract/reference; refresh the upstream command/API details when upgrading a tool.